

Automating the Application of HCI Principles

Skills for On-Demand UI Construction, the Human-AI Space to Think, and the Future of HCI

Nathan Conklin
nathan.conklin@vt.edu
Virginia Tech
Blacksburg, Virginia, USA

Miranda Capra
mcapra@gmail.com
Independent Consultant
Chapel Hill, North Carolina, USA

Chris North
north@cs.vt.edu
Virginia Tech
Blacksburg, Virginia, USA

Abstract

Human-computer interaction (HCI) is in the middle of a transition: large language models can now generate functional user interfaces (UIs) on demand from natural-language task descriptions. A user explains what they are trying to accomplish, and the system materializes a working interface to support it. This capability already exists in systems such as Claude and ChatGPT and continues to grow in fidelity as the underlying models improve. The next step along this trajectory is to move from interfaces that are merely generated to interfaces that are *generated well*. We propose a framework in which the dialogue between user and artificial intelligence (AI) becomes a Space to Think: a shared, structured cognitive workspace in which task decomposition produces an on-demand user interface as an extension of the user's thinking rather than as a separate artifact. Within this paradigm, classical HCI design knowledge (Nielsen's heuristics, Norman's affordance prescriptions, Web Content Accessibility Guidelines (WCAG) success criteria, cognitive-load constraints, and mixed-initiative principles) is encoded as skills: machine-readable skill.md files that the generating agent loads at runtime as software engineering tools. Skills turn HCI design knowledge into declarative, inspectable, version-controlled, and editable artifacts owned by the HCI community itself so that accessibility, learnability, and consistency become properties of a generative process rather than properties of a finished product. We outline a research agenda depicting a future where the HCI field transitions from today's design/knowledge heuristic checklist towards a future where the craft becomes machine-readable, executable, and open.

CCS Concepts

• **Human-centered computing** → **User interface design; Accessibility**; • **Software Engineering** → **Automatic programming**;

ACM Reference Format:

Nathan Conklin, Miranda Capra, and Chris North. 2026. Automating the Application of HCI Principles: Skills for On-Demand UI Construction, the Human-AI Space to Think, and the Future of HCI. In *Proceedings of The Future of HCI Workshop (FutureHCI '26)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FutureHCI '26, Blacksburg, VA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/XX
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

HCI has been dominated by a common design workflow: a designer anticipates a user's task, creates the layout of the corresponding user interface, and ships the result as an application. The user's job is to translate intent into the vocabulary the designer chose. When the user's task does not match the designer's anticipation, the user pays the cost in workarounds, rework, and abandoned features. Direct manipulation [12] made this within user's reach; the web made it ubiquitous; mobile platforms made it global. HCI primarily remains in a regime of pre-designed interaction. Three converging shifts now make this regime contingent rather than necessary. First, large language models (LLMs) can generate high-fidelity, executable user interface code from natural language with growing reliability [4]. Second, agent architectures based on chain-of-thought, ReAct, Tree-of-Thoughts, and Plan-and-Solve prompting allow models to decompose tasks into sub-goals and act through tools [5]. Third, commercial systems including Anthropic's Artifacts, OpenAI's Canvas, and Vercel's v0 enable dialogue surfaces in which an interface materializes while the user is still describing what they want [2]. The component pieces of an on-demand user interface paradigm are already in production; what is missing is a conceptual frame that explains what they are doing.

HCI-empowered software engineering will enable on-demand User Interface construction within a Human-AI cognitive workspace. The dialogue between the user and AI will become more than a query-answer transaction; it will become a Space to Think [1][3] for building shared context and common consensus through interaction. User capabilities will grow through Human-AI conversation. We make the following contributions: (i) We define the Space to Think as a structured cognitive workspace and show how task decomposition naturally lives inside it. (ii) We describe an architecture for on-demand user interface construction as a workspace rather than a separate artifact. (iii) We show that classical HCI design principles can be encoded as procedural constraints on the generator, turning learnability, accessibility, and consistency into properties of a process rather than properties of a product. (iv) We describe a framework with an agenda for the next year and the next quarter-century towards the evaluation, predictability, and accessibility of dynamically generated content.

1 summarizes the framework and serves as the structural backbone of the paper: (1) the Space to Think and its dialogue decomposition; (2) the shared task model; (3) the generation pipeline; (4) automatic HCI principles enforcement via skills; (5) the generated user interface as an extension of thinking; (6) bidirectional coupling between dialogue and user interface.

Our primary contribution is the identification of a concrete mechanism for procedural HCI: skills, packaged as machine-readable

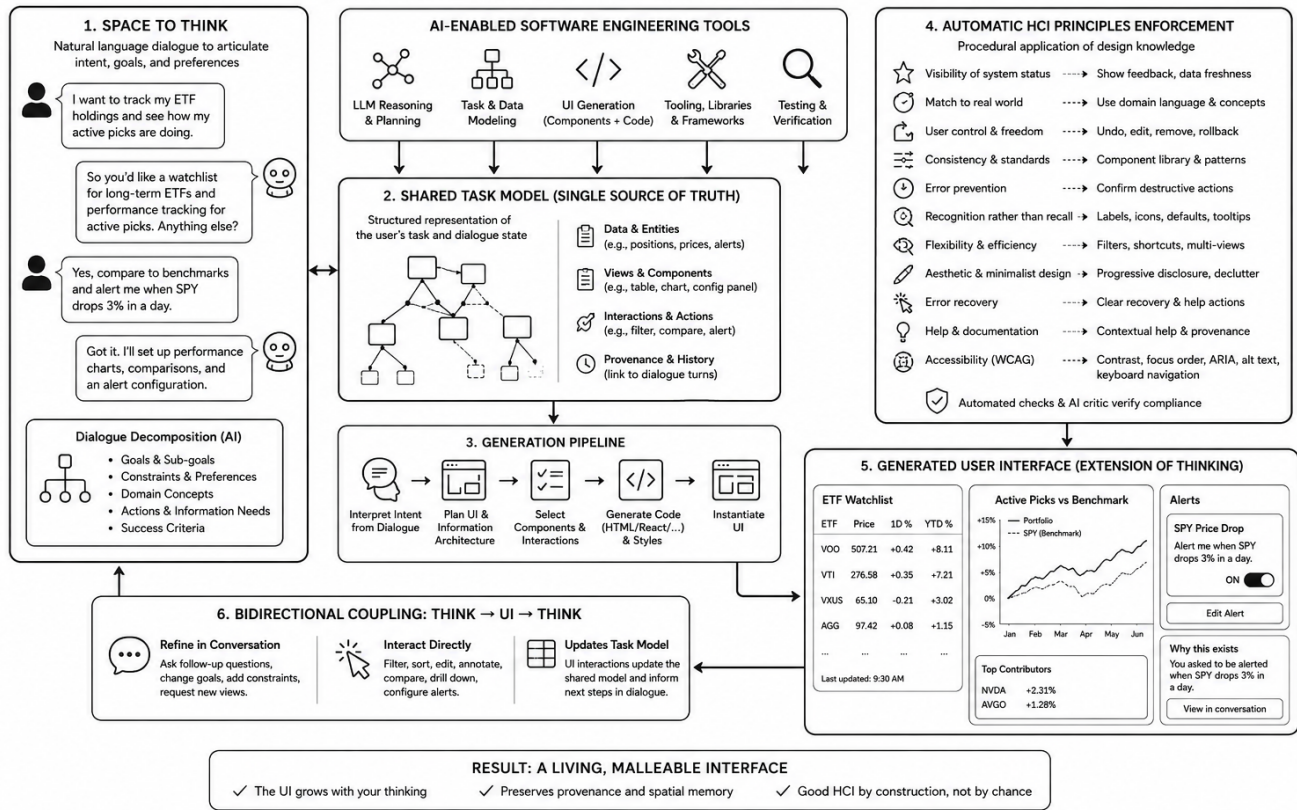


Figure 1: AI-Enabled Software Engineering That Automates UI Construction and the Application of HCI Principles

skill.md files that the AI loads at runtime as software engineering tools. The **skill.md** construct enables HCI design knowledge to become a declarative, inspectable, version-controlled, and editable software engineering tool managed by the HCI community itself. Each skill encodes HCI design knowledge including Nielsen's heuristics [10], Norman's affordance prescriptions [11], WCAG success criteria [6], cognitive-load constraints, and mixed-initiative principles [8] as procedural guidance that the agent can read, follow, and verify against its own output. Where heuristic evaluation has historically been a post-hoc audit performed by humans, skills make it an inline constraint on generation. Good HCI becomes a property of a process, not a property of a product.

2 Generative UI and LLM-Driven User Interface Construction

Generative UI, user interface synthesis by LLMs, has rapidly evolved. Recent systems, such as JELLY, generate user interfaces around task-driven data models that users reshape through natural language and direct manipulation [4]. Generative UI systems utilize LLMs to construct task-specific user interfaces instead of returning linear text, enabling cognitive gains on structured and information-dense tasks. Kyungho Lee summarizes this emerging area with five characteristics of generative UI: computational co-creation, expanded design-space exploration, representational fluidity, contextual adaptation, and synthesis-over-selection [9]. Parallel work including

Design2Code, UICoder [14], and retrieval-augmented GUI generation demonstrates improving fidelity in LLM-driven user interface capabilities. Many industry capabilities have been made available on the market. Vercel's v0 operationalized text-to-React interface generation; Anthropic's Artifacts and OpenAI's Canvas introduced side-by-side conversational workspaces in which documents, code, and user interfaces are constructed directly from dialogue [2]. In parallel, the malleable software community argues that software should remain continuously reshapeable by users rather than fixed after deployment. Earlier model-based UI systems demonstrated that abstract task and dialogue models could generate interfaces across heterogeneous devices; LLM-based generation may be understood as a stochastic continuation of this model-driven tradition.

Mixed-initiative interaction provides the conceptual foundation for these systems. Horvitz's Principles of Mixed-Initiative User Interfaces [8] articulated how autonomous assistance and user control should be balanced under uncertainty, while the TRAINS project demonstrated integrated dialogue-driven planning systems combining natural-language understanding and task reasoning. Grosz and Sidner's discourse theory further introduced linguistic, intentional, and attentional structures that map naturally onto collaborative cognitive workspaces. Contemporary LLM systems inherit many of these ideas while embedding them inside end-to-end conversational environments.

3 On-Demand UI as Cognitive Extension

A Space to Think is a shared, persistent, structured workspace in which a human user and an AI collaborator externalize, manipulate, and refine the representations needed to solve a task [1][3]. The space is shared: both participants can read and write all of its contents, and both can hold attention on the same fragment. The space is structured: it is a representation with internal scaffolding of sub-goals, hypotheses, evidence, decisions, and tools-under-construction. Finally, the space is operational for the user. This third property is what distinguishes a Space to Think from an ordinary chat log and what allows an on-demand UI to enable new user interface capabilities and expand the Human-AI shared context.

Decomposition in the Space to Think provides a mixed-initiative interaction [8]: either the human or the AI may propose a sub-goal, contest it, or revise the plan, and uncertainty is treated as a primary signal for when the AI should ask rather than act. Each sub-goal that is identified via dialogue produces a corresponding affordance in the user interface, so the structure of the user's thought is mirrored by the structure of the user interface tool that emerges. The on-demand UI is not the output of the Space to Think; it is an extension of it. The user interface is constructed as a part of a shared context that enables the user to work, think, and through use provide feedback to the AI to support future concept iterations. Dialogue state and UI state share the same context that is grounded in both a task-driven data model and model-based UI development [4].

4 Automating HCI Design Principles

User interface construction can be automated while incorporating critical design principles, applied procedurally to every generated artifact to achieve a coverage that hand design rarely matches in practice.

Heuristics as procedural constraints: Nielsen's ten heuristics [10] (visibility of system status, match between system and the real world, user control and freedom, consistency and standards, error prevention, recognition rather than recall, flexibility and efficiency, aesthetic and minimalist design, error recovery, and help and documentation) become constraints for user interface design. UI generation can be trained or prompted to satisfy these constraints, and downstream testing can verify them, much as UICoder uses compiler and vision-language feedback in a self-training loop [14].

Affordances and signifiers by construction: Norman's distinction between perceived affordances and conventions [11] prescribes a critical design principle: do not generate a control whose visual form does not signal its action. A button must look pressable; a draggable handle must look draggable; an editable field must signal editability. UI generation can enforce these conventions at the component-library level, validating the design before it reaches the user.

Cognitive load and progressive disclosure: Cognitive load theory [13] motivates a UI generation policy of minimum cognitive footprint. The user interface exposes the capability the first time the user's dialogue articulates the functional need. Progressive disclosure shifts from a design-time concern to an artifact of user interaction and UI construction.

Direct manipulation and the conversation-manipulation tradeoff: Shneiderman's three direct-manipulation criteria [12] become an affordance for design. The user interface ensures visible objects of interest, rapid reversible incremental actions, and replacement of command syntax with object manipulation.

Accessibility as a generator invariant: Automation of accessibility ensures WCAG conformance [6]. UI generation can treat WCAG success criteria as hard constraints: every component emitted must pass automated checks for contrast, focus order, ARIA (accessible rich internet application) roles, alt text, keyboard reachability, and screen reading. LLM-driven generation will make accessibility affordable across arbitrary domains, aligned with ability-based personalization.

Learnability as scaffolded emergence: As the UI generation grows with the user's articulation of need, the user quickly learns what each component is for: because they asked for it. This collapses the classical learnability problem. The interface functions become learnable through its provenance: every affordance carries a back-link to the dialogue that justified it, providing the recognition-over-recall property [10] for free.

5 Skills as Software Engineering Tools for HCI

A **skill** is a procedural specification of a capability, packaged in a directory containing a `skill.md` file and any supporting assets (schemas, scripts, templates, example components). The `skill.md` is human-readable Markdown with structured frontmatter declaring when the skill applies (a triggering description), what the AI must do (procedural steps and constraints), and what counts as success (verification criteria). Skills sit alongside the other AI-enabled software engineering tools (Figure 1, top center) the agent draws on inside the Space to Think. Skills are declarative, inspectable, and editable: an HCI researcher can write or audit a skill without retraining a model, and the agent will load it on demand whenever the triggering description matches the current task. We propose the integration of HCI design knowledge repackaged as a small, opinionated library of skills the agent invokes during UI generation (Figure 1, panel 4):

- **heuristics-nielsen.skill.md** encodes Nielsen's ten usability heuristics [10] as a checklist the generator must satisfy: every action produces visible feedback; vocabulary matches the user's domain (extracted from the dialogue); every destructive action is reversible; primary actions are recognizable from location and label without prior exposure; layouts obey a consistent component library.
- **affordances-norman.skill.md** encodes Norman's prescription [11] that perceived form must signal real action. A button must look pressable; a draggable handle must look draggable; an editable field must signal editability. Because the generator emits structured component code rather than raw pixels, signifier compliance is enforced at the component-library level: any control failing the check is rejected before it reaches the user.
- **accessibility-wcag.skill.md** encodes WCAG success criteria [6] as hard constraints. Every emitted component must pass automated checks for contrast, focus order, ARIA roles,

alt text, and keyboard reachability. Conformance becomes a generator invariant rather than a release-time audit.

- **ability-based-personalization.skill.md** encodes per-user accommodations beyond WCAG conformance, in the tradition of SUPPLE [7]. Based on a user profile that the user articulates, this skill modifies UI construction and the other skills application. Personalization may include larger targets, longer dwell, simpler gestures, amplified contrast, type scaling, simplified layouts, plain-language labels, reduced surface area, longer time budgets, and step-by-step disclosure. This is uniquely enabled by the on-demand paradigm: each user interface is generated for one user.
- **cognitive-load-progressive-disclosure.skill.md** encodes a minimum sufficient surface policy. The user interface exposes only the affordances the dialogue has so far justified; new capability appears the first time the user's articulated need entails it.
- **mixed-initiative.skill.md** encodes Horvitz's principles [8] for the agentic affordances in the user interface: every action that affects the world is instrumented with confirmation, reversibility, an audit trail, and a visible confidence signal.
- **direct-manipulation.skill.md** encodes Shneiderman's three criteria [12]: visible objects of interest, rapid reversible incremental actions, and replacement of command syntax with object manipulation, applied to each UI component.

The skills library is not a fixed set; HCI researchers contribute new skills as research advances; practitioners audit, fork, and extend them; organizations write skills encoding domain-specific conventions and obtain a generator that follows them. Each artifact of the HCI subdiscipline (design pattern, accessibility lint rule, layout heuristic, etc.) becomes a runnable deliverable rather than a paper hoping to influence practitioners. Skills carry properties that hand design lacks:

- **Uniform.** Every component in every generated UI is checked, not only those a designer happened to remember.
- **Inspectable.** A user can ask why a control behaves the way it does and the agent can cite the skill it followed. The same back-link supports the "Why this exists" affordance in panel 5.
- **Editable.** Updates ship at the speed of HCI research, not the speed of pretraining. A regulator, an accessibility advocate, or a peer reviewer can read the skill and judge whether it captures the standard.
- **Verifiable.** The same Markdown that instructs the generator can be compiled into automated tests. Conformance becomes part of HCI rather than a subjective audit.

6 Future Work and Impact

HCI knowledge changes at the speed of HCI research; whereas updating model weights take months. Skills can be authored, reviewed, version-controlled, and deployed in hours. Skills provide auditability; a **skill.md** is a reviewable text file. Organizations and individuals can layer skills without forking the underlying model. On-demand UI provides a cognitive extension of the Space to Think, and a natural mechanism for turning HCI design knowledge into

procedural constraints on generation. Good HCI becomes a property of a process, not of a product; the field's craft becomes runnable, inspectable, and shareable.

LLM-generated code is notoriously difficult to verify, and the same is true of the user interfaces it produces. This is not a new defect introduced by automated generation but a consequence of HCI's longstanding validation problem that even human-designed interfaces resist proof of "good design". Current practice relies on user studies, formative and summative evaluations, and heuristic walkthroughs that are expensive, slow, and still inconclusive on learnability, error resistance, and long-term usability. Future work is needed to develop formal validation criteria, automated heuristic and accessibility checks that are machine interpretable. The development of predictive models for cognitive load and learnability will enable the ability to automate the validation of generated user interfaces. We anticipate that verification skills will emerge as a knowledge base alongside generation skills, becoming a future executable validation toolkit.

Our research team will extend this work over the next year by open-sourcing our initial skills library and applying it within our ongoing Space to Think research to enable on-demand UIs that surface, explore, and externalize chain-of-thought reasoning in Human-AI interaction. At a five-year horizon, we expect HCI to have a skills ecosystem comparable to today's component libraries: open repositories, organizational forks, peer-reviewed contributions. A new HCI paper will routinely ship a skill alongside its findings, and reviewers will expect a runnable artifact. The unit of design-knowledge transfer will shift from the heuristic checklist to the executable skill. At a ten-year horizon, we anticipate that pre-designed UIs will be rare for heterogeneous, idiosyncratic tasks. Accessibility will become a default that every user receives as the UI generation leverages an accessibility-wcag skill and validates conformance. At a twenty-five-year horizon, the most consequential change will be conceptual. Cognitive properties will be assembled, and design will be the discipline of shaping that assembly. The skills library may become the field's accumulated craft: machine-readable, executable, and open. We may look back on the era of the fixed application the way we now look back on the era of the form letter.

Acknowledgments

Thanks to Na Meng for your review and support of this paper. Thanks to Dave Cadorette and Vince Traglia for your support over the last two years enabling the exploration of this topic.

References

- [1] Christopher Andrews, Alex Endert, and Chris North. 2010. Space to think: large high-resolution displays for sensemaking. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Atlanta, Georgia, USA) (CHI '10). Association for Computing Machinery, New York, NY, USA, 55–64. doi:10.1145/1753326.1753336
- [2] Anthropic. 2024. *Artifacts are now generally available*. <https://www.anthropic.com/news/artifacts>
- [3] Anthropic. 2026. *Claude is a space to think*. Retrieved 2026-02-04. from <https://www.anthropic.com/news/claude-is-a-space-to-think>
- [4] Yining Cao, Peiling Jiang, and Haijun Xia. 2025. Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (CHI '25). Association for Computing Machinery, New York, NY, USA, Article 686, 20 pages. doi:10.1145/3706598.3713285
- [5] Shuaihang Chen, Yuanxing Liu, Wei Han, Weinan Zhang, and Ting Liu. 2025. A Survey on LLM-based Multi-Agent System: Recent Advances and New Frontiers

- in Application. arXiv:2412.17481 [cs.CL] <https://arxiv.org/abs/2412.17481>
- [6] World Wide Web Consortium. 2024. *Web Content Accessibility Guidelines (WCAG) 2.2*. Retrieved 2026-05-20 from <https://www.w3.org/TR/WCAG22/>
- [7] Krzysztof Z. Gajos, Daniel S. Weld, and Jacob O. Wobbrock. 2010. Automatically generating personalized user interfaces with Supple. *Artificial Intelligence* 174, 12 (2010), 910–950. doi:10.1016/j.artint.2010.05.005
- [8] Eric Horvitz. 1999. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Pittsburgh, Pennsylvania, USA) (CHI '99). Association for Computing Machinery, New York, NY, USA, 159–166. doi:10.1145/302979.303030
- [9] Kyungho Lee. 2025. Towards a Working Definition of Designing Generative User Interfaces. In *Companion Publication of the 2025 ACM Designing Interactive Systems Conference (DIS '25 Companion)*. Association for Computing Machinery, New York, NY, USA, 489–495. doi:10.1145/3715668.3736365
- [10] Jakob Nielsen. 1994. *Heuristic evaluation*. John Wiley & Sons, Inc., USA, 25–62.
- [11] Donald A. Norman. 2013. *The Design of Everyday Things* (revised and expanded ed.). Basic Books, New York, NY, USA.
- [12] Shneiderman. 1983. Direct Manipulation: A Step Beyond Programming Languages. *Computer* 16, 8 (1983), 57–69. doi:10.1109/MC.1983.1654471
- [13] John Sweller. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science* 12, 2 (1988), 257–285. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1207/s15516709cog1202_4 doi:10.1207/s15516709cog1202_4
- [14] Jason Wu, Eldon Schoop, Alan Leung, Titus Barik, Jeffrey P. Bigham, and Jeffrey Nichols. 2024. UICoder: Finetuning Large Language Models to Generate User Interface Code through Automated Feedback. arXiv:2406.07739 [cs.CL] <https://arxiv.org/abs/2406.07739>

A Figure 1 Detailed Description

A.1 Overview

The figure shows a cycle of 6 panels.

- (1) Space to think → panel 2
- (2) Shared task model → panels 1, 3
- (3) Generation pipeline → panel 5
- (4) Automatic HCI principles enforcement → panel 5
- (5) Generated user interface → panel 6
- (6) Bi-directional coupling → panel 1

There are two additional boxes.

- AI-enabled software engineering tools → panel 2
- Result (below the entire diagram)

A.2 Panel 1. Space to Think

Natural language dialogue to articulate intent, goals, and preferences.

- User: I want to track my ETF holdings and see how my active picks are doing.
- Response: So you'd like a watchlist for long-term ETCs and performance tracking for active picks. Anything else?
- User: yes, compare to benchmarks and alert me when SPY drops 3% in a day.
- Response: Got it. I'll set up performance charts, comparisons, and an alert configuration

Dialogue Decomposition (AI)

- Goals & Sub-Goals
- Constraints & Preferences
- Domain Concepts
- Actions & Information Needs
- Success Criteria

Panel 1 → “Panel 2. Shared Task Model”.

A.3 AI-Enabled Software Engineering Tools

- LLM Reasoning & Planning
- Task & Data Modeling
- UI Generation (Components + Code)
- Tooling, Libraries, & Frameworks
- Testing & Verification

AI-Enabled Software Engineering Tools → “Panel 2. Shared Task Model”.

A.4 Panel 2. Shared Task Model

Single source of truth.

Structured representation of the user's task and dialogue state.

- Data & Entities (e.g., positions, prices, alerts)
- Views & Components (e.g., table, chart, config panel)
- Interactions & Actions (e.g., filter, compare, alert)
- Provenance & History (link to dialogue turns)

Panel 2 → “Panel 3. Generation Pipeline”.

A.5 Panel 3. Generation Pipeline

- Interpret Intent from Dialogue
- Plan UI & Information Architecture
- Select Components & Interactions
- Generate Code (HTML/React/...) & Styles
- Instantiate UI

Panel 3 → “Panel 5. Generated User Interface”.

A.6 Panel 4. Automatic HCI Principles Enforcement

Procedural application of design knowledge.

- Visibility of system status → Show feedback, data freshness
- Match to real world → Use domain language & concepts
- User control & freedom → Undo, edit, remove, rollback
- Consistency & standards → Component library & patterns
- Error prevention → Confirm destructive actions
- Recognition rather than recall → Labels, icons, defaults, tooltips
- Flexibility & efficiency → Filters, shortcuts, multi-views
- Aesthetic & minimalist design → Progressive disclosure, declutter
- Error recovery → Clear recovery & help actions
- Help & documentation → Contextual help & provenance
- Accessibility (WCAG) → Contrast, focus order, ARIA, alt text, keyboard navigation

Automated checks & AI critic verify compliance.

Panel 4 → “Panel 5. Generated User Interface”.

A.7 Panel 5. Generated User Interface

Extension of thinking.

This panel has a dashboard with four boxes of information.

- **ETF Watchlist:** a table with four columns (ETF, Price, 1D%, YTD %) and rows for four different stock symbols (VOO, VTI, VXUS, AGG). Last updated at 9:30 AM.
- **Active Picks vs Benchmark:** a line graph with two lines, Portfolio and SPY (Benchmark). The lines start at 0% in January and rise through June, with a small dip in April. The Portfolio rises faster, ending at +10% compared to Benchmark which ends at +5%. Top contributors are NVDA +2.31% and AVGO +1.28%.
- **Alerts:** SPY Price Drop (alert me when SPY drops 3% in a day) toggle switch is ON. Button “Edit Alert”.
- **Why this exists:** you asked to be alerted when SPY drops 3% in a day. Button “View in conversation”.

A.8 Panel 6. Bi-Directional Coupling

Think → UI → Think

- **Refine in Conversation:** Ask follow-up questions, change coals, add constraints, request new views.
- **Interact Directly:** Filter, sort, edit, annotate, compare, drill down, configure alerts.
- **Updates Task Model:** UI Interactions update the shared model and inform next steps in dialogue.

A.9 Result

A living, malleable interface.

- The UI grows with your thinking
- Preserves provenance and spatial memory
- Good HCI by construction, not by chance